

Documento de Apoio: Arquitetura e Implementação do Sistema de Arquivos ZFS

Disciplina: Sistemas Operacionais

Curso: Engenharia da Computação - UFC

Equipe: Iuri Fernandes Bastos, Matheus Parente Reis, Vinicius Alexandre Gomes, Yuri Fontenele Mamede

1. Introdução

O ZFS (Zettabyte File System) foi concebido originalmente em 2005 pela Sun Microsystems com a ideia de redefinir a forma como os sistemas operacionais fazem o gerenciamento do armazenamento de dados. De modo diferente das abordagens tradicionais, onde o sistema de arquivos atua de maneira isolada sobre um gerenciador de volumes lógicos (LVM) ou sobre controladores RAID de hardware, o ZFS propõe unificar essas camadas. Tal unificação resolve o clássico problema da "falha de comunicação" entre o sistema de arquivos e os discos, permitindo que o sistema tenha "consciência" total da topologia física do armazenamento. O panorama do projeto mudou em 2010 com a aquisição da Sun pela Oracle Corporation, que tornou o código do ZFS proprietário. Como resposta a essa privatização, a comunidade e os desenvolvedores originais uniram-se para criar um *fork* de código aberto e independente, resultando no projeto OpenZFS, que atualmente mantém e distribui o sistema de arquivos de forma multiplataforma.

2. Arquitetura de Armazenamento ZFS

A arquitetura do ZFS abandona a ideia de partições fixas. Ao invés disso, o modelo pool de armazenamento é utilizado de forma que o espaço seja distribuído de forma dinâmica. A hierarquia bem definida a seguir, serve como base para a estrutura de dados:

Discos Físicos: As unidades de hardware (HDDs, SSDs, NVMe).

Vdevs (Virtual Devices): Os componentes fundamentais de construção do ZFS. Um Vdev agrupa os discos físicos e é a camada onde a redundância (como RAID-Z ou espelhamento) é aplicada. Em um caso onde o Vdev falhe por completo, todo o pool associado será perdido.

Zpool: O pool de armazenamento agregado. Ele agrupa um ou mais Vdevs, somando suas capacidades. As operações de escrita são distribuídas automaticamente entre os Vdevs que compõem o pool.

Datasets e Zvols: A interface final de consumo. Datasets atuam como diretórios que compartilham o espaço livre do zpool, podendo possuir cotas e reservas. Zvols são volumes de bloco que são frequentemente usados para virtualização e alvos iSCSI.

3. Mecanismos de Integridade e Confiabilidade

O ZFS foi concebido perante a premissa de que o hardware irá falhar de maneira imprevisível em algum momento. Haja vista essa inevitabilidade, para garantir integridade absoluta da informação, serão utilizados dois pilares tecnológicos centrais:

Copy-on-Write (CoW)

Ao contrário dos sistemas *overwrite-in-place* (que sobrescrevem o dado antigo no disco), o modelo transacional CoW do ZFS grava as modificações em um novo bloco livre. Após a confirmação bem-sucedida da gravação, os ponteiros da árvore de metadados serão atualizados em uma operação atômica. Isso impossibilita a corrupção do sistema de arquivos no caso de uma queda de energia durante uma escrita por exemplo.

Árvores de Merkle e Auto-Healing

Cada bloco de dados gravado possui um *checksum* (hash criptográfico de 256 bits). Esse *checksum* não é armazenado no próprio bloco, mas sim no bloco pai que aponta para ele, criando uma Árvore de Merkle até o bloco raiz do pool chamado de uberblock. Sempre que um dado é lido, o ZFS recalcula o hash de maneira instantânea. Se houver divergência (corrupção silenciosa de dados), o sistema detecta a anomalia e, utilizando a redundância dos Vdevs (RAID-Z ou espelhamento), reconstrói o dado correto de forma transparente (Auto-healing) e repara o bloco corrompido em disco.

4. Níveis de Redundância (RAID-Z)

O ZFS implementa o próprio RAID por software. O RAID-Z foi projetado especificamente para resolver o *Write Hole* do RAID de hardware, eliminando a necessidade de controladores excessivamente caros. Abaixo, uma tabela comparativa dos principais modelos de organização de Vdevs:

Configuração (Vdev)	Descrição da Paridade	Tolerância a Falhas	Casos de Uso Comuns
Mirror	Cópia exata dos dados em múltiplos discos.	1 ou mais discos por espelho.	Máximo desempenho de IOPS, bancos de dados.

Configuração (Vdev)	Descrição da Paridade	Tolerância a Falhas	Casos de Uso Comuns
RAID-Z1	Paridade única distribuída. Similar ao RAID 5.	1 disco por Vdev.	Armazenamento geral de baixo custo.
RAID-Z2	Dupla paridade distribuída. Similar ao RAID 6.	2 discos por Vdev.	O padrão para servidores corporativos.
RAID-Z3	Tripla paridade distribuída. Exclusividade do ZFS.	3 discos por Vdev.	Arquivos massivos e backups de longo prazo.

5. Gerenciamento de Memória e Cache

Um dos motivos para a alta curva de aprendizado e alto uso de hardware do ZFS reside em sua complexa gestão de memória. Ele não utiliza o cache de página padrão do kernel, mas sim a utilização de uma arquitetura própria para aumentar a velocidade de operações de I/O.

ARC (Adaptive Replacement Cache): Fica alocado na memória RAM do sistema. Diferente dos caches LRU, o ARC balanceia os blocos acessados recentemente e os acessados frequentemente (MFU), ajustando de forma dinâmica a proporção para maximizar as taxas de acerto. Ele possui um alto consumo de RAM disponível.

L2ARC (Level 2 ARC): Uma extensão do ARC focada em dispositivos mais rápidos que os discos principais, como SSDs ou NVMe, usado para dados predominantemente de leitura e acessos aleatórios.

ZIL (ZFS Intent Log) e SLOG: Quando há uma escrita síncrona (onde o aplicativo exige confirmação imediata), o ZFS utiliza o ZIL para registrar transações temporárias antes de gravá-las no disco principal. Um dispositivo rápido dedicado (SLOG - Separate Intent Log) pode absorver essas escritas, melhorando a latência drasticamente.

6. Otimizações de Espaço: Compressão e Deduplicação

Alocação baseada em blocos aliada ao processamento robusto permite otimizações importantes para data centers modernos:

Compressão Nativa Transparente: Suporta algoritmos modernos como LZ4 e ZSTD.

Operando em tempo real, os dados são comprimidos antes da gravação em disco. Na maioria dos casos, isso aumenta o desempenho global do sistema e aumenta enquanto economiza espaço, pois a CPU atual comprime mais rápido do que os discos rodam para efetuar a gravação.

Deduplicação (Block-level Dedup): Mantém uma Tabela de Deduplicação (DDT) baseada nos hashes de cada bloco. Evita a gravação de blocos duplicados. No entanto, consome intensamente memória RAM (recomenda-se algo em torno de 1 a 5 GB de RAM por cada 1 TB de dados deduplicados armazenados).

7. Snapshots, Clones e Replicação

A verdadeira flexibilidade do modelo **Copy-on-Write** se mostra na geração de **Snapshots**.

Um Snapshot do ZFS custa tempo e espaço iniciais iguais a zero. Como blocos novos não substituem blocos antigos, criar um Snapshot apenas congela o estado atual da árvore de blocos e protege aqueles ponteiros para eles não serem excluídos. Os blocos modificados no futuro passam a ser cobrados contra o tamanho daquele Snapshot.

Os comandos nativos `zfs send` e `zfs receive` permitem serializar esses snapshots e enviá-los de forma incremental para outra máquina pela rede de forma incrivelmente ágil, servindo de base nativa para backups de altíssima fidelidade e planos de recuperação de desastres.

8. Considerações Finais

O ZFS representa uma profunda quebra nos paradigmas de design de armazenamento que antecedem o ano de 2005. Ao unir camadas que antes eram desconectadas, possibilita um controle refinado com garantias rigorosas de dados corretos e validados. Embora possua desvantagens em ambientes que sejam limitados por memória RAM, flexibilidade severa de reestruturação de Vdevs ou hardwares incompatíveis adequadamente com acesso direto à memória, o ZFS é o atual padrão industrial em sistemas de armazenamento crítico voltados para segurança cibernética e robustez em virtualização corporativa.

9. Referências Bibliográficas e Fontes

OpenZFS Project. *OpenZFS Documentation*. Documentação técnica, arquitetura ARC e guias de uso mantidos pela comunidade. Acesso no ano de 2026.

The FreeBSD Project. *Chapter 23. The Z File System (ZFS)*. FreeBSD Handbook. Referência em comportamento ZFS Unix, Vdevs e Tuning. Acesso no ano de 2026.

iXsystems. *ZFS Primer*. TrueNAS Documentation Hub. Manual primário utilizado sobre os pilares, confiabilidade, cache e escalabilidade do ZFS em produção corporativa. Acesso no ano de 2026.